

ContractIQ

Contract Clause QA with Corrective Retrieval

Client overview · Prepared for sharing · Raunak Rath

1. Executive Summary

ContractIQ helps users ask natural-language questions over legal and contract documents. It retrieves relevant clauses, grades whether those passages are good enough to answer, and only then generates a response. When retrieval quality is weak, the system automatically retries with a broader search before answering.

Primary users: Legal ops, contract analysts, research teams reviewing MSAs/NDAs.

2. Key Features

- Upload PDF or TXT contracts
- Ask clause-level questions in plain English
- Passage quality grades: Correct / Ambiguous / Incorrect
- Corrective actions: Accept, Re-retrieve, or Web-search (stub)
- Dashboard with query volume and correction rate
- Query history and upcoming-feature roadmap in the UI

3. How the System Works (Code Flow)

The application follows a simple layered design: React dashboard → FastAPI backend → core processing module → SQLite storage.

1. User uploads a contract in the web dashboard.
2. Backend extracts text (PDF via PyMuPDF, or plain TXT).
3. Document is split into chunks and indexed for search.
4. User asks a question; top passages are retrieved.
5. Each passage is graded for relevance to the question.
6. If quality is mixed or poor, the system re-retrieves with an expanded query.
7. An answer is generated only from the filtered passages.
8. Grades, corrective action, and answer are shown and saved to history.

4. Technology Stack

- Frontend: React + Vite + Tailwind (port 5171)
- Backend: FastAPI + SQLite (port 8001)
- AI: embeddings + NLI grader; Ollama optional; demo mode without GPU

5. How to Run

```
cd contractiq/backend && DEMO_MODE=1 uvicorn main:app --reload --port 8001
cd contractiq/frontend && npm install && npm run dev
Open http://127.0.0.1:5171
```

6. Sample Files for Demo

- samples/sample_msa.txt — Master Services Agreement (TXT)
- samples/sample_msa.pdf — Same agreement as PDF

- samples/sample_nda.txt — Non-Disclosure Agreement
- Try asking: What is the termination notice period?

7. Main API Endpoints

- POST /api/upload — upload contract
- POST /api/query — ask a question
- GET /api/documents — list uploads
- GET /api/history — past Q&A;
- GET /api/stats — dashboard metrics

8. Upcoming Features

The following capabilities are planned for future releases and are shown in the product UI under Upcoming.

- Adaptive question-complexity routing
- Live web-search recovery when local context is insufficient
- Benchmark evaluation on public QA datasets
- Latency and cost comparison vs plain RAG

9. Folder Layout

- contractiq/frontend — web dashboard
- contractiq/backend — API and core logic
- contractiq/samples — demo upload / paste files
- contractiq/docs — internal report and slides